

Why ZenVoice Misses Words — and How to Fix It

*A measured diagnosis of English speech-recognition accuracy,
with a prioritised remediation plan.*

PREPARED FOR	Yash Chaudhary
DATE	25 July 2026
BRANCH	codex/zenvoice-figma-redesign
TEST MACHINE	Apple M5 · 24 GB · macOS 27.0
ENGINE	whisper.cpp v1.9.1 (Metal + flash-attention)
STATUS	Issue confirmed and reproduced

THE ONE-LINE ANSWER

Yes, the problem is real — but speaking fast is *not* the main cause. The biggest single source of errors is that ZenVoice transcribes your speech in small disconnected pieces and never listens to the whole sentence at once.

1 Executive Summary

Your report — “ZenVoice can't identify some words, and speaking fast makes it worse” — was reproduced under controlled conditions. Speaking speed does hurt accuracy, but it is the *third* most important factor. Two other problems matter more, and one of them is entirely of our own making.

What was found

RANKED CAUSES OF MISSED WORDS

#	CAUSE	PLAIN-ENGLISH DESCRIPTION	ACCURACY COST
1	Chopped-up transcription	ZenVoice cuts your speech at every pause and transcribes each piece separately. Words at the cuts get mangled.	+8 pts
2	Model ladder gap	The smaller AI models collapse when you speak quickly, and the app recommends one of them to most Macs.	+11 pts
3	Noise leaking as text	Silence and background sound occasionally get typed out as words like <code>you</code> or <code>(static)</code> .	Occasional

“Accuracy cost” means percentage points of extra word errors. If a system normally gets 3 words wrong out of 100, adding 8 points makes it 11 out of 100.

WHAT THIS MEANS FOR YOU SPECIFICALLY

Your machine is already set to **Whisper Medium (English)** — one of the strongest models available. That means cause #2 does not apply to you. On clean whole-sentence audio, your model made **zero errors** in testing. Every error you are currently experiencing is almost certainly coming from cause #1: the chopping.

What was ruled out

Two fixes that sound obviously correct were tested and **rejected on the evidence**. Feeding the previous sentence back into the AI as context made accuracy roughly **twice as bad**. Boosting quiet microphone audio before transcription made almost no difference. Both are documented in Section 5 so they are not attempted again later.

The plan

Four changes are proposed in Section 6, in priority order. The first one alone addresses the largest cause, takes the least code, and is expected to remove most of the errors you are seeing today.

2 How This Was Tested

Everything in this report is measured, not assumed. A purpose-built test harness was written that drives the exact same speech engine and the exact same settings that ZenVoice uses in production.

The method, step by step

- 1 **Built a benchmark harness.** A standalone program that loads `whisper.cpp` with ZenVoice's production settings — greedy decoding, timestamps disabled, GPU and flash-attention on — copied directly from `WhisperTranscriber.swift`.
- 2 **Created a controlled test set.** Four sentences deliberately loaded with the kind of vocabulary that trips up dictation: technical terms (*middleware*, *idempotent*, *Kubernetes*, *backfill*), proper nouns (*Priya*, *Postgres*, *Marathi*) and easily-confused word pairs (*bearer* vs *barrier*, *pull request* vs *full request*).
- 3 **Recorded each sentence at three speeds.** 170 words per minute (relaxed), 280 wpm (brisk) and 380 wpm (genuinely fast). Twelve clips in total.
- 4 **Simulated real microphone conditions.** Studio-clean audio flatters any speech engine. Each clip was also tested quietened to roughly a tenth of full volume with a light noise floor added — closer to a laptop microphone across a desk.
- 5 **Scored every result automatically.** Word Error Rate was computed against the known reference text, ignoring punctuation and capitalisation so only genuine word mistakes are counted.

WHAT "WORD ERROR RATE" MEANS

Word Error Rate (WER) answers one question: out of every 100 words you speak, how many come out wrong? A word counts as wrong if it is replaced by a different word, dropped entirely, or invented out of nowhere.

- 0–3% — feels flawless. Roughly one slip per long paragraph.
- 5–10% — noticeable. About one wrong word per two sentences.
- 15%+ — frustrating. You spend more time fixing than dictating.
- 30%+ — unusable. Nearly one word in three is wrong.

Honest limitation

The test speech was generated by macOS text-to-speech, not recorded from a human. Synthetic speech is cleaner and more evenly paced than real speech, so the absolute percentages in this report are almost certainly **optimistic** — real-world numbers will be worse. What synthetic speech does give is perfect repeatability: every configuration heard byte-identical audio, so the *comparisons* between them are trustworthy. Treat the rankings and the gaps as solid, and the absolute figures as a floor rather than a forecast.

3 Does Speaking Fast Actually Break It?

Short answer: yes — but only on the smaller AI models. Speaking speed is an amplifier of an underlying weakness, not an independent problem.

WORD ERROR RATE BY SPEAKING SPEED · REALISTIC MICROPHONE CONDITIONS

SPEAKING SPEED	WHISPER BASE (148 MB)	WHISPER MEDIUM (1.5 GB)	LARGE-V3-TURBO Q5 (547 MB)
170 wpm — relaxed	5.1%	1.9%	3.2%
280 wpm — brisk	10.3%	5.2%	3.2%
380 wpm — fast	33.7%	7.3%	8.5%
Degradation, slow → fast	6.6× worse	3.8× worse	2.7× worse

Reading this table in plain English

On the small **Base** model, going from relaxed to fast speech takes you from “occasional slip” to **one word in three being wrong**. One clip failed so completely that 95% of its words were wrong — the model essentially gave up. That is the experience a user on a 16 GB Mac gets today, because that is the model the app recommends to them.

On **Medium** and **Turbo**, fast speech still costs accuracy, but it degrades gracefully — from near-perfect to merely good. The cliff disappears.

The analogy: *a small model is like a listener who is competent but not fluent in the topic. At conversational pace they follow you fine. Speed up, and they lose the thread completely. A larger model is a listener who genuinely knows the subject — speed them up and they still catch nearly everything, because they can predict where the sentence is going.*

Conclusion: your instinct was right that speed matters. But the fix is not to slow down your speech — it is to make sure the model underneath is strong enough that speed stops mattering.

4 The Three Confirmed Issues

Issue 1 — ZenVoice never listens to your whole sentence

HIGHEST IMPACT

Where: AppDelegate.swift · acceptStablePhrase and completeLiveRecording · Affects: every user (live preview is on by default)

What happens. While you dictate, ZenVoice watches for pauses. Every time you pause, it grabs the audio since the last pause, transcribes that fragment on its own, and glues the resulting text onto what it already has. When you release the key, the final text handed to your app is that glued-together string. The complete recording is saved to disk — but it is **never transcribed as a single piece**.

The analogy: imagine handing a translator your speech through a letterbox, two seconds at a time, and never letting them hear the full sentence. They will get each scrap roughly right — and mangle every single word that happens to straddle a cut.

The evidence. The same audio, the same model, decoded two ways:

WHOLE-RECORDING VS. PIECE-BY-PIECE TRANSCRIPTION

MODEL	TRANSCRIBED WHOLE	TRANSCRIBED IN PIECES (WHAT ZENVOICE DOES)	DAMAGE
Whisper Base	5.1%	11.1%	+6.0 pts
Whisper Medium — your model	0.0%	8.4%	+8.4 pts
Large-v3-Turbo Q5	2.3%	7.9%	+5.6 pts

Read the middle row carefully. Given the whole sentence, your model made **no mistakes at all**. Given the same sentence in fragments, it got 8 words in 100 wrong. The model is not failing — the plumbing around it is.

What it looks like in practice — real outputs from the test run:

WHOLE	the sidocar proxy keeps retrying idempotent requests
IN PIECES	the sidocar pro- He keeps retrying idempotent requests

WHOLE	before the backfill completes which would lock the whole table
IN PIECES	before the backfill completes. which would lock the whole table

WHOLE	so it validates the bearer token before it queries the Postgres replica
IN PIECES	so it val- the bearer token before it queries the Postgres reference

Why speaking fast makes this worse. Fast speech runs words together, so ZenVoice's pause detector fires in less predictable places — often mid-phrase rather than at a natural boundary. More cuts in worse places means more mangled words.

Issue 2 — The model ladder has a hole in the middle

HIGH IMPACT · NOT YOU

Where: `VerifiedModelCatalog.swift` · `ModelRecommendations.swift` · Affects: users on Tiny or Base — not your current setup

What happens. ZenVoice offers three model sizes: Tiny (75 MB), Base (148 MB) and Medium (1.5 GB). That is a ten-fold jump from Base to Medium with nothing in between. Meanwhile the recommendation logic picks a tier purely from how much memory the Mac has: under 12 GB gets Tiny, 12–20 GB gets Base, above that gets Medium.

The consequence is that a perfectly capable 16 GB Apple Silicon Mac is steered to **Base** — the model that hits 33.7% error at speed — when the same machine could comfortably run something far better.

ACCURACY VS. DOWNLOAD SIZE · REALISTIC MICROPHONE CONDITIONS, ALL SPEEDS

MODEL	SIZE	WORD ERROR RATE	LANGUAGES	IN CATALOG?
Whisper Base (English)	148 MB	15.5%	English only	Yes
Whisper Medium (English)	1 530 MB	4.7%	English only	Yes
Large-v3-Turbo Q5	547 MB	4.7%	Multilingual	No — missing

The headline. Large-v3-Turbo matches Whisper Medium's accuracy exactly, at roughly **one third of the download size**, and it handles every language rather than English alone. It is the obvious default for this app and it is not currently offered. Whisper Small (466 MB) is likewise missing and would fill the Base-to-Medium gap.

The multilingual point matters beyond raw accuracy: the Hinglish and Indian-language work already in `docs/hinglish` and `docs/LANGUAGES.md` cannot be served by an English-only model at all.

Issue 3 — Silence sometimes gets typed out as words

LOW IMPACT · EASY FIX

Where: `TranscriptCleaner.swift` line 8 · Affects: anyone dictating in a room with background noise

What happens. When Whisper is handed a very short scrap of near-silence — which ZenVoice's pause-based chopping produces regularly — it sometimes invents text rather than returning nothing. In testing, near-silent audio produced the outputs `you`, `(static)` and `[BLANK_AUDIO]`.

ZenVoice's cleanup pass strips anything in **square** brackets, so `[BLANK_AUDIO]` is caught correctly. But `(static)` uses round brackets and `you` has no markers at all — so both are treated as real speech and typed into whatever app you are using.

The analogy: *the filter is a bouncer with a list of one disguise. Anyone wearing a different hat walks straight in.*

This is cosmetically the most annoying failure — a stray “you” appearing in the middle of your document reads as a bug even though the underlying transcription of your actual speech was fine.

5 Fixes That Were Tested and Rejected

Both of these sound obviously correct. Both were measured. Both are documented here specifically so nobody wastes time re-implementing them in six months.

Rejected — feeding the previous text back as context

The idea. If chopping loses context, give each fragment the previously transcribed text as a hint. Whisper supports exactly this via an “initial prompt”.

What actually happened. Accuracy got **dramatically worse** — from 11.1% error to 26.3%, roughly a doubling. Whisper stopped transcribing and started *autocompleting*: given plausible preceding text, it invented fluent continuations that were never spoken.

YOU SAID	...fully localized for Hindi Tamil and Marathi speakers
IT TYPED	...fully localized for Hindi Tamil. The new flow is also available in Marathi speakers

An entire clause was fabricated. For a dictation tool, invented text is far more dangerous than a missing word — you may not notice it. **Verdict: do not do this.**

Rejected — boosting quiet microphone audio

The idea. Quiet input clearly hurts accuracy, so normalise the volume up before handing audio to the engine.

What actually happened. Error rate moved from 15.5% to 14.1% — statistically close to nothing — and it made **three of the twelve clips actively worse**. The reason is that Whisper already normalises loudness internally as part of converting audio to its input representation. Boosting beforehand mostly just amplifies the noise floor alongside the speech. **Verdict: not worth the code.**

Partially useful — beam search decoding

Switching from fast “greedy” decoding to more thorough “beam search” decoding helped the small model meaningfully (15.8% → 12.8%) at about 30% more processing time. On Medium and Turbo it changed **nothing at all** — those models are already confident enough that exploring alternatives finds no better answer. Worth enabling for small models only, never globally.

SUMMARY OF REJECTED AND CONDITIONAL APPROACHES

APPROACH	BEFORE	AFTER	VERDICT
Carry previous text as context	11.1%	26.3%	Reject — causes fabrication
Normalise quiet audio	15.5%	14.1%	Reject — negligible
Beam search on small model	15.8%	12.8%	Adopt for small models only
Beam search on Medium / Turbo	4.7%	4.7%	Skip — pure cost, no gain

6 Proposed Solutions, in Priority Order

Ordered by accuracy gained per unit of work. Priority 1 alone should remove most of the errors you are experiencing today.

PRIORITY 1 Transcribe the whole recording once, when you release the key

What changes. The live, piece-by-piece transcription stays exactly as it is — but it gets demoted to what it should always have been: a *preview*. It shows you words as you speak so the app feels responsive. The moment you release the dictation key, ZenVoice transcribes the complete recording it already saved to disk, in one pass, and *that* result is what gets typed into your app.

HOW THIS HELPS, IN PLAIN ENGLISH

Right now the AI is asked to understand your sentence through a letterbox. This change lets it hear the entire sentence before deciding what you said — so words at the cut points stop being guesses. In testing this took your model from **8.4% word errors down to 0%** on the same audio.

Crucially, this costs you almost nothing in speed. The full recording is already saved. Re-transcribing 20 seconds of speech on your M5 takes about **0.2 seconds** — you will not perceive it.

The one complication. If “commit on pause” is switched on, text has already been inserted into your app before you release the key. The final result then has to be reconciled with what is already on screen rather than simply appended. The safe path is to ship this for the default configuration first — where nothing has been inserted yet — and handle the commit-on-pause case as a follow-up with a diff-and-replace.

EXPECTED GAIN

–8 points WER

EFFORT

Small — one code path

RISK

Low

PRIORITY 2 Add Large-v3-Turbo and Whisper Small to the model catalog

What changes. Two models are added to the verified catalog: **Large-v3-Turbo Q5** (547 MB, multilingual) and **Whisper Small** (466 MB). Turbo becomes the recommended default for any Apple Silicon Mac. The recommendation logic stops deciding purely on installed memory and takes GPU transcription speed into account as well.

HOW THIS HELPS, IN PLAIN ENGLISH

Today a 16 GB Mac is told to use the Base model, which gets roughly **one word in three wrong** when its owner speaks quickly. Turbo delivers the same accuracy as the 1.5 GB Medium model in a third of the download, so there is no longer any reason to send anyone to a weak model. For those users this is a drop from **15.5% to 4.7%** word errors — the single largest accuracy jump available.

It also unlocks the Hindi and Hinglish work already planned. Turbo is multilingual; the English-only models physically cannot do that job.

Note. This does not change anything for you directly — you are already on Medium. It matters for everyone else, and it lets you eventually swap 1.5 GB of disk for 547 MB at identical accuracy.

EXPECTED GAIN

–11 points WER
for affected users

EFFORT

Small — catalog entries

RISK

Low

PRIORITY 3 Teach the transcript cleaner to recognise non-speech

What changes. The cleanup pass is extended to drop round-bracket annotations such as `(static)` and `(wind blowing)`, and to discard a small set of known filler outputs — `you`, `Thank you.` — but only when they are the *entire* content of a fragment, never when they appear inside real speech.

HOW THIS HELPS, IN PLAIN ENGLISH

It stops the most visibly embarrassing failure: a random “you” or “(static)” appearing in the middle of your document when you paused to think. This does not make the AI smarter — it just stops obvious garbage reaching your screen.

The “entire fragment only” rule is what keeps it safe: if you genuinely say “thank you” in a sentence, it survives untouched.

EXPECTED GAIN

Removes visible artifacts

EFFORT

Very small

RISK

Low — needs care

PRIORITY 4 Enable beam-search decoding for small models only

What changes. When a Tiny or Base model is selected, the engine switches from greedy decoding to beam search with a beam width of 5. Medium and Turbo keep the current fast path.

HOW THIS HELPS, IN PLAIN ENGLISH

Greedy decoding commits to the most likely word at every step and never looks back. Beam search keeps several candidate sentences alive and picks whichever reads best overall — so a wrong early guess can still be corrected. That safety net matters for a weak model and is redundant for a strong one.

Measured benefit: **15.8% → 12.8%** for Base, for about 30% more processing time. Zero measurable benefit on Medium or Turbo, which is precisely why it must be gated rather than switched on globally.

EXPECTED GAIN

−3 points WER
small models only

EFFORT

Very small

RISK

Low

7 Suggested Order of Work

A sequence that puts a measurement safety-net in place before changing anything, so every subsequent change can be proven rather than hoped for.

- 1 **Turn the benchmark harness into a permanent project target.** The harness written for this investigation currently lives outside the repository. Promoting it to something like `ZenVoiceAccuracyChecks`, alongside the existing `ZenVoiceRuntimeChecks`, means every future change to transcription can be measured instead of guessed at. This is the highest-leverage first step even though it changes no user-facing behaviour.
- 2 **Ship Priority 1** — whole-recording transcription on release, for the default configuration only. Measure before and after with the harness.
- 3 **Ship Priority 3** — the transcript cleaner fix. Small, independent, immediately visible improvement.
- 4 **Ship Priority 2** — the new catalog entries and revised recommendation logic. Larger surface area because it touches downloads, verification checksums and settings UI.
- 5 **Ship Priority 4** — beam search gated to small models. Trivial once the harness exists to confirm it does not regress anything.
- 6 **Handle the commit-on-pause reconciliation** left over from Priority 1, closing the last gap.

WHAT SUCCESS LOOKS LIKE

After Priorities 1 and 2, a user on a mid-range Mac speaking quickly should move from roughly **one word in three wrong** to **under one word in twenty**. For you specifically — already on a strong model — Priority 1 alone should take the visible error rate close to zero on clean input.

What is deliberately not proposed

- **Cloud transcription.** Would improve accuracy, but ZenVoice's local-only privacy guarantee is a core product promise. Out of scope.
- **Custom model fine-tuning.** Large effort, uncertain payoff, and unnecessary while the architectural issue in Priority 1 remains unfixed.
- **Rewriting the pause detector.** Once the whole recording is transcribed on release, the pause detector only governs the preview — its precision stops affecting final accuracy. Fixing Priority 1 makes this work unnecessary.

8 Appendix

A. Full per-clip results

WORD ERROR RATE PER CLIP · QUIET MICROPHONE WITH NOISE FLOOR · PRODUCTION DECODE SETTINGS

CLIP	SPEED	BASE	MEDIUM	TURBO Q5
1 · authentication middleware	170 wpm	0.0%	0.0%	0.0%
1 · authentication middleware	280 wpm	12.0%	4.0%	0.0%
1 · authentication middleware	380 wpm	4.0%	8.0%	4.0%
2 · Kubernetes sidecar	170 wpm	9.1%	0.0%	9.1%
2 · Kubernetes sidecar	280 wpm	13.6%	9.1%	9.1%
2 · Kubernetes sidecar	380 wpm	95.5%	13.6%	22.7%
3 · beta ship criteria	170 wpm	4.0%	4.0%	0.0%
3 · beta ship criteria	280 wpm	12.0%	4.0%	0.0%
3 · beta ship criteria	380 wpm	24.0%	4.0%	0.0%
4 · migration review	170 wpm	7.4%	3.7%	3.7%
4 · migration review	280 wpm	3.7%	3.7%	3.7%
4 · migration review	380 wpm	11.1%	3.7%	7.4%
Overall (word-weighted)		15.5%	4.7%	4.7%

The 95.5% outlier on clip 2 at 380 wpm is not a measurement error. The Base model produced a near-total failure on that clip — the kind of result a user would describe as “it just didn't understand me at all”.

B. Test sentences used

1. “Please refactor the authentication middleware so it validates the bearer token before it queries the Postgres replica and returns unauthorized instead of a server error.”
2. “The Kubernetes cluster is throttling because the sidecar proxy keeps retrying idempotent requests against a stale endpoint slice in the staging namespace.”
3. “Ship the beta on Thursday only if the crash rate stays low and the onboarding flow is fully localized for Hindi Tamil and Marathi speakers.”
4. “I asked Priya to review the pull request but she said the migration script drops the index before the backfill completes which would lock the whole table.”

C. Glossary

Word Error Rate (WER)

Out of every 100 words spoken, how many come out wrong — substituted, dropped or invented. Lower is better.

Percentage points (pts)

The arithmetic difference between two percentages. Going from 3% to 11% is a rise of 8 percentage points, not 8 percent.

Greedy vs. beam search decoding

Greedy picks the single most likely next word and never reconsiders. Beam search keeps several candidate sentences alive and chooses the best complete one. Beam search is slower and more accurate — but only when the model is uncertain enough for the alternatives to differ.

Initial prompt

Optional text given to Whisper before it transcribes, intended to bias it towards expected vocabulary. Tested here and found actively harmful when used to carry previous transcript text.

Quantisation (the “Q5” in Large-v3-Turbo Q5)

Storing the model's numbers at lower precision to shrink the file. Q5 cuts size substantially with negligible accuracy loss — which is how a 547 MB model matches a 1.5 GB one.

Words per minute (wpm)

Speaking pace. Ordinary conversation runs 140–160 wpm; confident dictation 200–250; 380 wpm is genuinely fast speech.

D. Reproducing these results

The benchmark harness is a standalone Swift package that links the same whisper.cpp framework as ZenVoice. It supports five measurement modes: decode-parameter comparison, chunk-versus-whole simulation, context-prompt evaluation, gain-normalisation evaluation, and short-segment hallucination probing. All twelve audio clips are generated deterministically from the four reference sentences via macOS text-to-speech, so the whole test set can be recreated from scratch. Promoting this harness into the repository is Priority 1 of the work sequence in Section 7.

All measurements taken 25 July 2026 on Apple M5 / 24 GB / macOS 27.0 using whisper.cpp v1.9.1 with Metal GPU acceleration and flash attention enabled — the same configuration ZenVoice ships. Decode parameters were copied verbatim from `Sources/ZenVoiceRuntime/WhisperTranscriber.swift`. Absolute figures are based on synthetic speech and should be read as a best case; comparisons between configurations are directly reliable because every configuration processed byte-identical audio.